

1895. Largest Magic Square

A $k \times k$ **magic square** is a $k \times k$ grid filled with integers such that every row sum, every column sum, and both diagonal sums are **all equal**. The integers in the magic square **do not have to be distinct**. Every 1×1 grid is trivially a **magic square**.

Given an $m \times n$ integer grid, return *the **size** (i.e., the side length k) of the **largest magic square** that can be found within this grid.*

Example 1:

7	1	4	5	6
2	5	1	6	4
1	5	4	3	2
1	2	7	3	4

Input: grid = [[7,1,4,5,6],[2,5,1,6,4],[1,5,4,3,2],[1,2,7,3,4]]

Output: 3

Explanation: The largest magic square has a size of 3.

Every row sum, column sum, and diagonal sum of this magic square is equal to 12.

- Row sums: $5+1+6 = 5+4+3 = 2+7+3 = 12$

- Column sums: $5+5+2 = 1+4+7 = 6+3+3 = 12$

- Diagonal sums: $5+4+3 = 6+4+2 = 12$

Example 2:

5	1	3	1
9	3	3	1
1	3	3	8

Input: grid = `[[5,1,3,1],[9,3,3,1],[1,3,3,8]]`

Output: 2

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 50$
- $1 \leq \text{grid}[i][j] \leq 10^6$

Explanation in Hindi

◆ Magic Square क्या होता है?

$k \times k$ Magic Square ऐसा square होता है जिसमें:

- हर row का sum बराबर हो
- हर column का sum बराबर हो
- दोनों diagonal का sum बराबर हो

👉 1×1 हमेशा magic square होता है।

◆ Problem क्या पूछ रहा है?

- एक $m \times n$ grid दी गई है
 - इसमें से सबसे बड़ा magic square ढूँढना है
 - और उसका size (k) return करना है
-

◆ Example 1

```
grid = [  
  [7,1,4,5,6],  
  [2,5,1,6,4],  
  [1,5,4,3,2],  
  [1,2,7,3,4]  
]
```

✅ Output: 3

Explanation

इस grid के अंदर 3×3 magic square मिलता है:

5 1 6

5 4 3

2 7 3

✓ Sum check:

- Row sums = 12
- Column sums = 12
- Diagonal sums = 12

👉 इसलिए answer = 3

◆ Example 2

```
grid = [  
  [5,1,3,1],  
  [9,3,3,1],  
  [1,3,3,8]  
]
```

✓ Output: 2

👉 यहाँ सबसे बड़ा magic square 2×2 है।

◆ Solution Approach (Logic)

Step 1:

सबसे पहले समझो:

- Maximum possible size = $\min(m, n)$
-

Step 2:

हम बड़े size से छोटे size की तरफ check करेंगे
(ताकि पहला valid मिलते ही answer मिल जाए)

$k = \min(m, n) \rightarrow 1$

Step 3:

हर possible $k \times k$ sub-grid के लिए:

1. पहली row का sum लो $\rightarrow$ targetSum
2. हर row का sum check करो
3. हर column का sum check करो
4. दोनों diagonal का sum check करो

अगर सब बराबर $\Rightarrow$ magic square मिल गया 

Step 4:

जैसे ही कोई valid magic square मिले:

return k

अगर कुछ न मिले:

return 1

◆ Pseudo Flow (Easy)

for k = maxSize to 1:

 for each k×k subgrid:

 if all rows, columns, diagonals equal:

 return k

return 1

◆ Time Complexity

- Grid size $\leq 50 \times 50$
 - Brute force acceptable
 - Complexity $\approx O(n^4)$ (constraints के अंदर safe)
-

◆ Final Answer

Example Output

Example 1 **3**

Example 2 **2**

Code in java

```
class Solution {
```

```

public int largestMagicSquare(int[][] mat) {

    int R = mat.length;

    int C = mat[0].length;

    // Row prefix sums
    int[][] rowSum = new int[R][C];
    for (int r = 0; r < R; r++) {
        rowSum[r][0] = mat[r][0];
        for (int c = 1; c < C; c++) {
            rowSum[r][c] = rowSum[r][c - 1] + mat[r][c];
        }
    }

    // Column prefix sums
    int[][] colSum = new int[R][C];
    for (int c = 0; c < C; c++) {
        colSum[0][c] = mat[0][c];
        for (int r = 1; r < R; r++) {
            colSum[r][c] = colSum[r - 1][c] + mat[r][c];
        }
    }

    int maxSize = 1;

    // Try every cell as top-left
    for (int r = 0; r < R; r++) {
        for (int c = 0; c < C; c++) {
            int maxPossible = Math.min(R - r, C - c);

            // Try bigger squares first
            for (int size = maxPossible; size > maxSize; size--) {

```

```

        if (isMagic(r, c, size, mat, rowSum, colSum)) {
            maxSize = size;
            break;
        }
    }
}
return maxSize;
}

```

```

private boolean isMagic(
    int sr, int sc, int size,
    int[][] mat, int[][] rowSum, int[][] colSum) {

```

```

    // Target sum from first row
    int target = rowSum[sr][sc + size - 1]
        - (sc > 0 ? rowSum[sr][sc - 1] : 0);

```

```

    // Check rows
    for (int r = sr; r < sr + size; r++) {
        int sum = rowSum[r][sc + size - 1]
            - (sc > 0 ? rowSum[r][sc - 1] : 0);
        if (sum != target) return false;
    }

```

```

    // Check columns
    for (int c = sc; c < sc + size; c++) {
        int sum = colSum[sr + size - 1][c]
            - (sr > 0 ? colSum[sr - 1][c] : 0);
        if (sum != target) return false;
    }

```



```
// Main diagonal
int d1 = 0;
for (int k = 0; k < size; k++)
    d1 += mat[sr + k][sc + k];
if (d1 != target) return false;

// Anti-diagonal
int d2 = 0;
for (int k = 0; k < size; k++)
    d2 += mat[sr + size - 1 - k][sc + k];

return d2 == target;
}
}
```